

Data and Artificial Intelligence

Cyber Shujaa Program

Week 7 Assignment

Linear Regression Models

Student Name: Violet Joy

Student ID:cs-da01-25025

Table of Content

Data and Artificial Intelligence	1
Cyber Shujaa Program.....	1
Week 7 Assignment Linear Regression Models	1
Introduction	3
Tasks completed.....	4
2.1: Explore the Dataset.....	4
2.2: Prepare the Data	11
2.3: Build the Model.....	13
2.4: Evaluate the Model	15
2.5: Visualize the Results.....	17
2.6: Share Your Work.....	20
Conclusion.....	21

Introduction

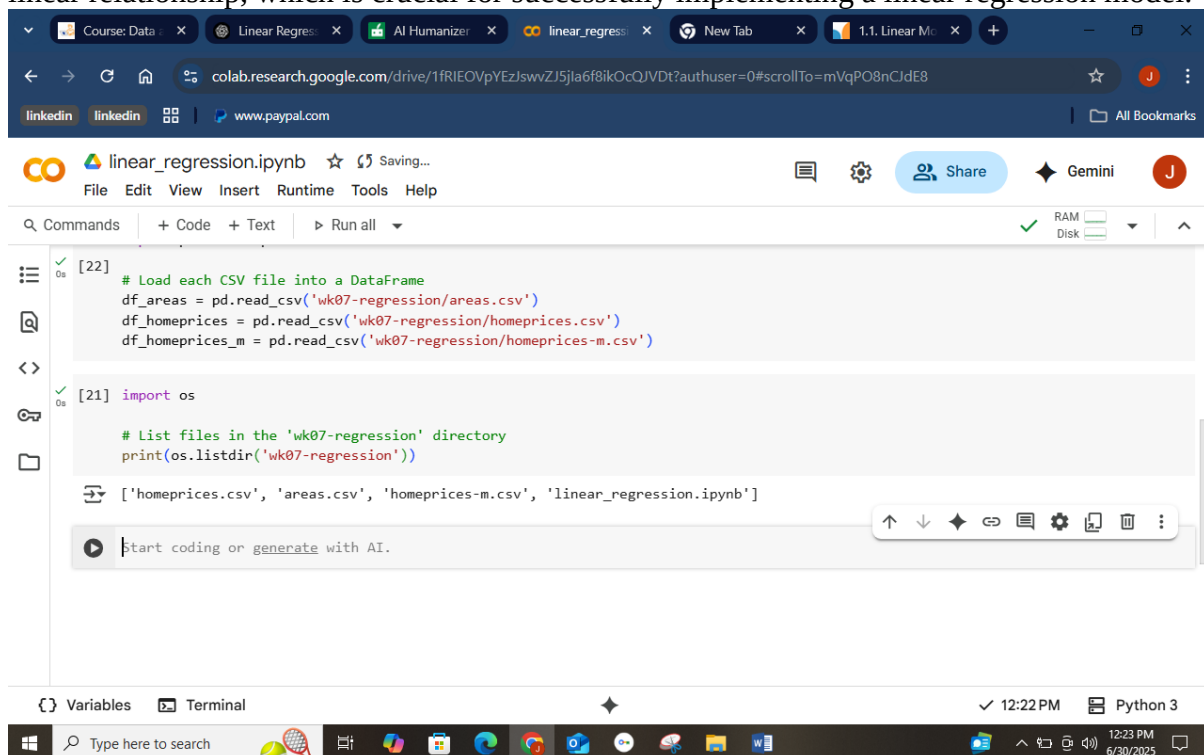
The objective of this project is to acquire hands-on experience with Python's Linear Regression. The primary objective is to use a full workflow, from data exploration to model evaluation, to apply basic machine learning concepts to a real-world dataset. Creating a basic linear regression model, separating data into training and testing sets, and evaluating the model's performance using accepted evaluation metrics are all part of the assignment. Furthermore, it is easier to interpret the model's behavior when predictions and regression lines are visualized. In addition to strengthening theoretical understanding, this practical exercise helps develop a portfolio of applied data science projects.

Tasks completed

2.1: Explore the Dataset

First, libraries like Pandas and NumPy are used to load the dataset into the Python environment. To comprehend its structure, basic exploration is carried out, which includes determining the data types, number of rows and columns, and existence of any missing values. The central tendencies and spread of the data can be seen with the aid of summary statistics like mean, median, minimum, and maximum values.

Scatter plots are used to visually analyze the relationship between the independent variable or variables and the dependent variable after the initial inspection. These plots aid in locating trends, patterns, or possible outliers in the data and offer information on the existence of a linear relationship, which is crucial for successfully implementing a linear regression model.



The screenshot displays a Google Colab notebook interface. The browser tabs at the top include 'Course: Data', 'Linear Regres...', 'AI Humanizer', 'linear_regres...', 'New Tab', and '1.1. Linear M...'. The address bar shows the Colab URL. The notebook title is 'linear_regression.ipynb'. The code editor shows two cells: Cell [22] loads CSV files into DataFrames, and Cell [21] lists files in the 'wk07-regression' directory. The output of Cell [21] is a list of files: ['homeprices.csv', 'areas.csv', 'homeprices-m.csv', 'linear_regression.ipynb']. The bottom status bar indicates 'Variables', 'Terminal', '12:22 PM', and 'Python 3'.

```
[22] # Load each CSV file into a DataFrame
df_areas = pd.read_csv('wk07-regression/areas.csv')
df_homeprices = pd.read_csv('wk07-regression/homeprices.csv')
df_homeprices_m = pd.read_csv('wk07-regression/homeprices-m.csv')

[21] import os

# List files in the 'wk07-regression' directory
print(os.listdir('wk07-regression'))

['homeprices.csv', 'areas.csv', 'homeprices-m.csv', 'linear_regression.ipynb']
```

Fig 2.1.1; Output of importing the dataset to google colab

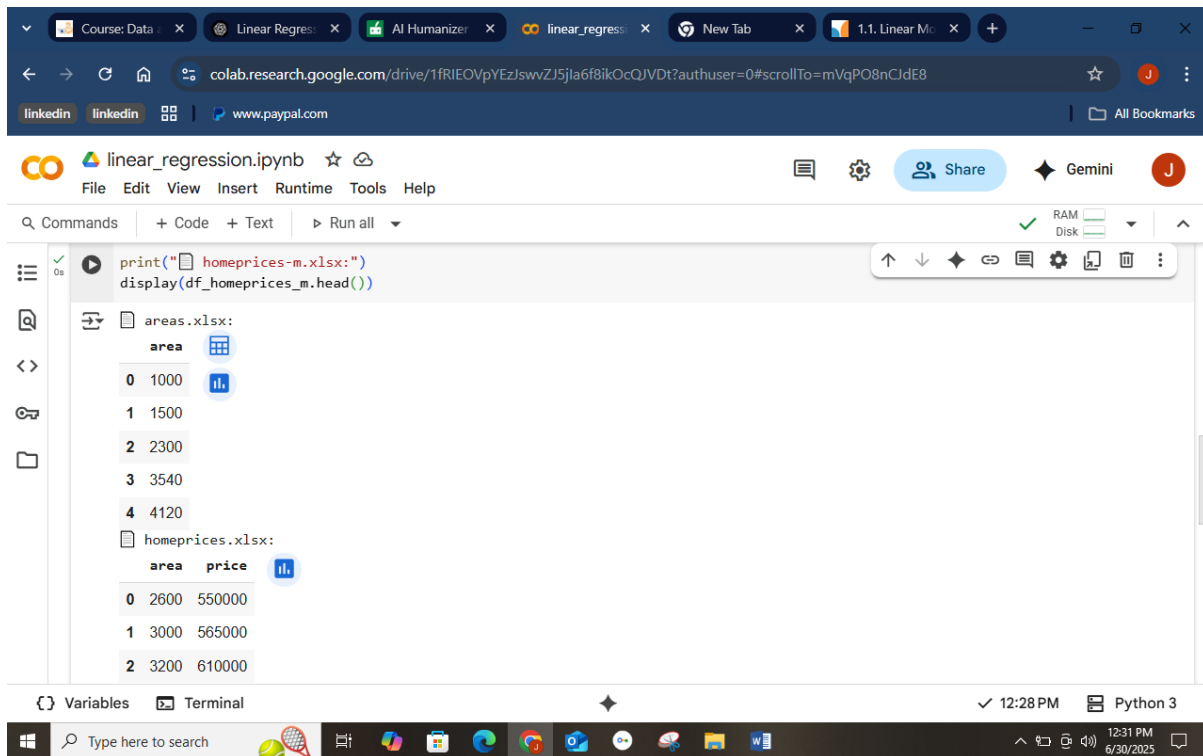


Fig 2.1.2: Output of displaying first few rows of areas and homeprices

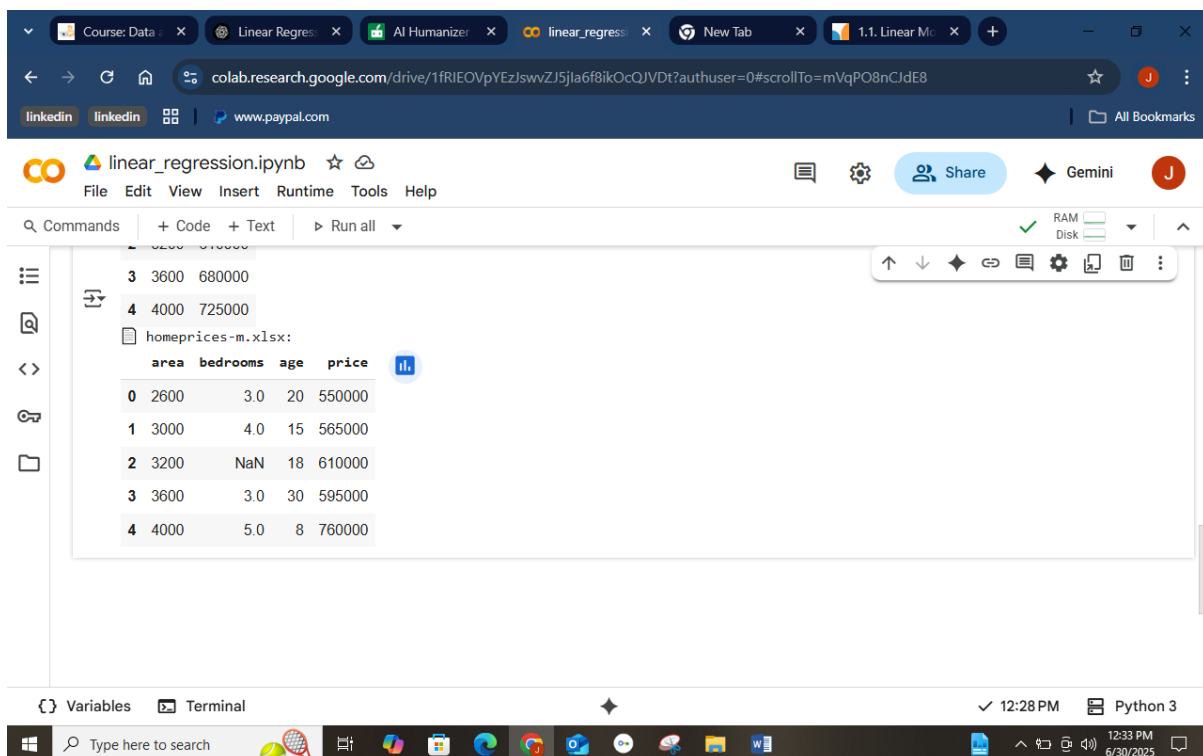
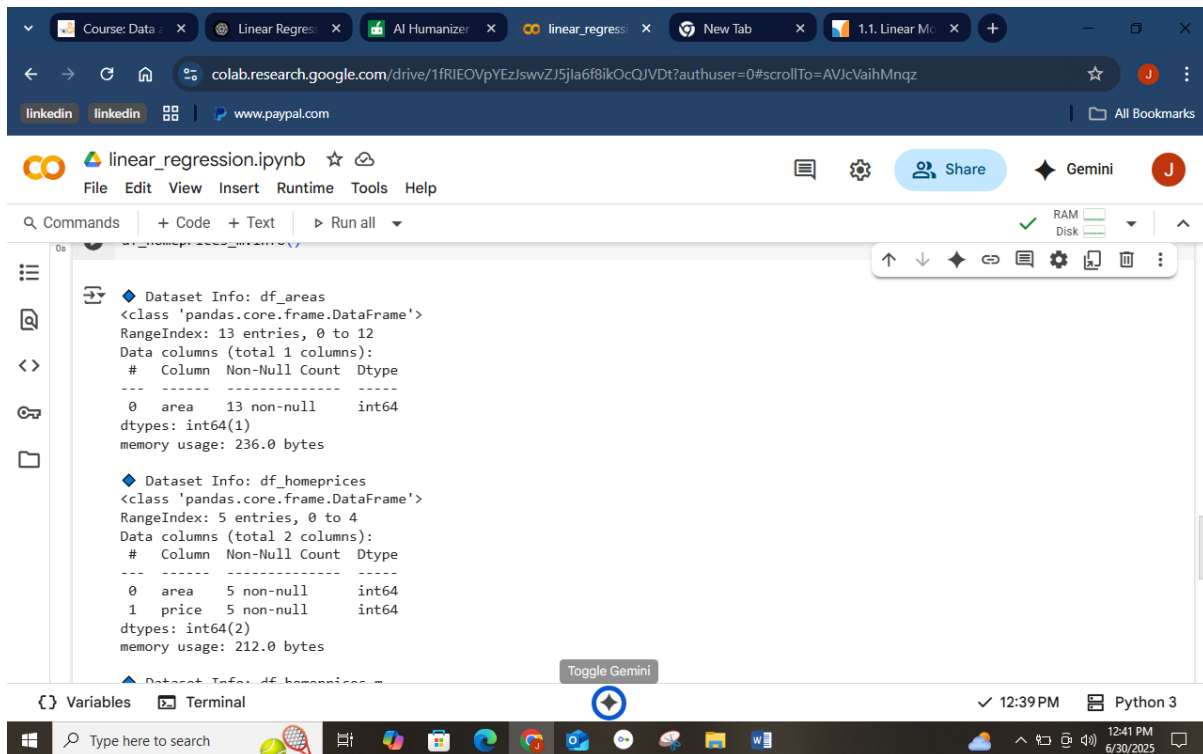


Fig 2.1.3: Output of displaying the first few rows of homeprices-m



The screenshot shows a Google Colab notebook titled 'linear_regression.ipynb'. The code cell displays the following output:

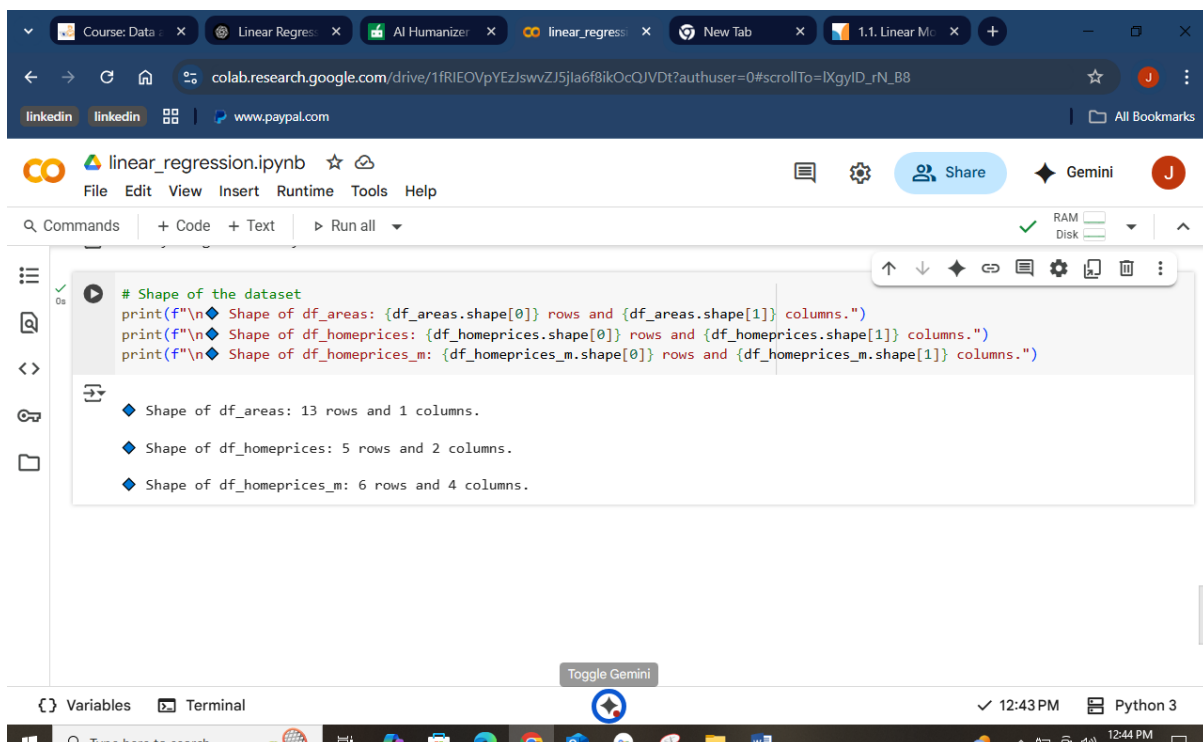
```

Dataset Info: df_areas
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 13 entries, 0 to 12
Data columns (total 1 columns):
#   Column  Non-Null Count  Dtype
---  -
0    area    13 non-null        int64
dtypes: int64(1)
memory usage: 236.0 bytes

Dataset Info: df_homeprices
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 2 columns):
#   Column  Non-Null Count  Dtype
---  -
0    area    5 non-null        int64
1    price   5 non-null        int64
dtypes: int64(2)
memory usage: 212.0 bytes

```

Fig 2.1.4: Output of checking the structure of the datasets



The screenshot shows a Google Colab notebook titled 'linear_regression.ipynb'. The code cell displays the following output:

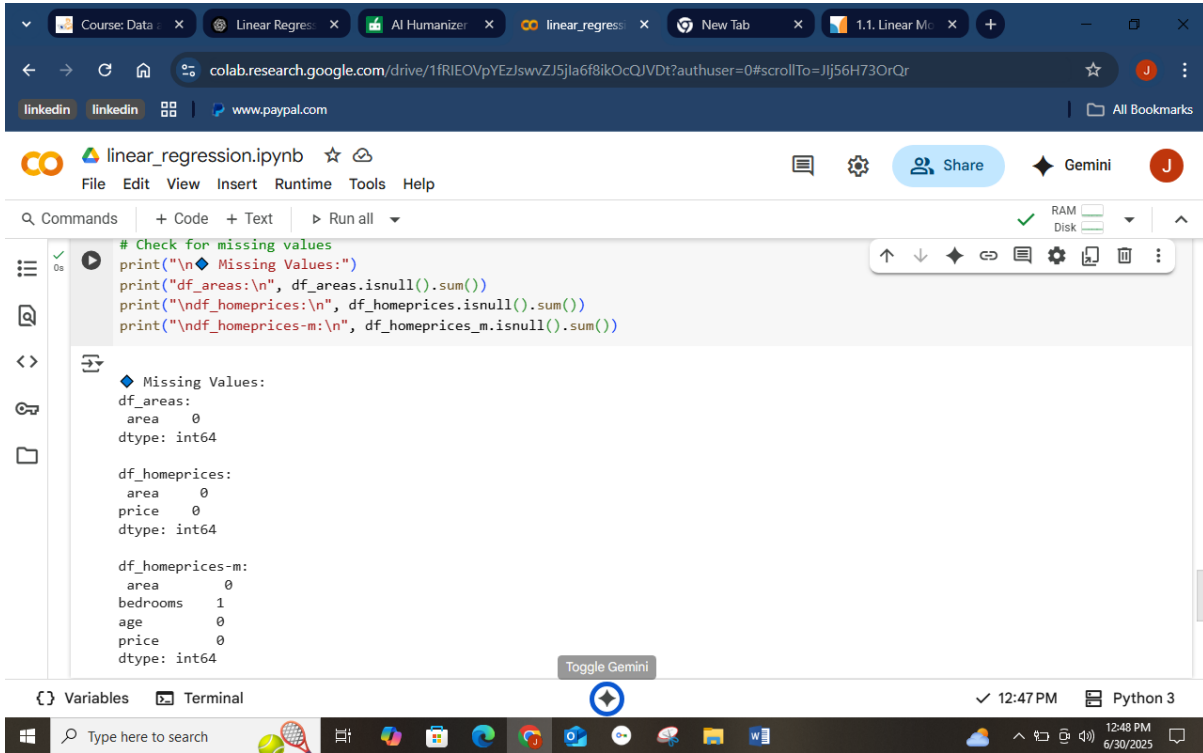
```

# Shape of the dataset
print(f"Shape of df_areas: {df_areas.shape[0]} rows and {df_areas.shape[1]} columns.")
print(f"Shape of df_homeprices: {df_homeprices.shape[0]} rows and {df_homeprices.shape[1]} columns.")
print(f"Shape of df_homeprices_m: {df_homeprices_m.shape[0]} rows and {df_homeprices_m.shape[1]} columns.")

Shape of df_areas: 13 rows and 1 columns.
Shape of df_homeprices: 5 rows and 2 columns.
Shape of df_homeprices_m: 6 rows and 4 columns.

```

Fig 2.1.5: Output of the shape of the dataset



The screenshot shows a Google Colab notebook titled "linear_regression.ipynb". The code cell contains the following Python code:

```
# Check for missing values
print("\n◆ Missing Values:")
print("df_areas:\n", df_areas.isnull().sum())
print("\ndf_homeprices:\n", df_homeprices.isnull().sum())
print("\ndf_homeprices-m:\n", df_homeprices_m.isnull().sum())
```

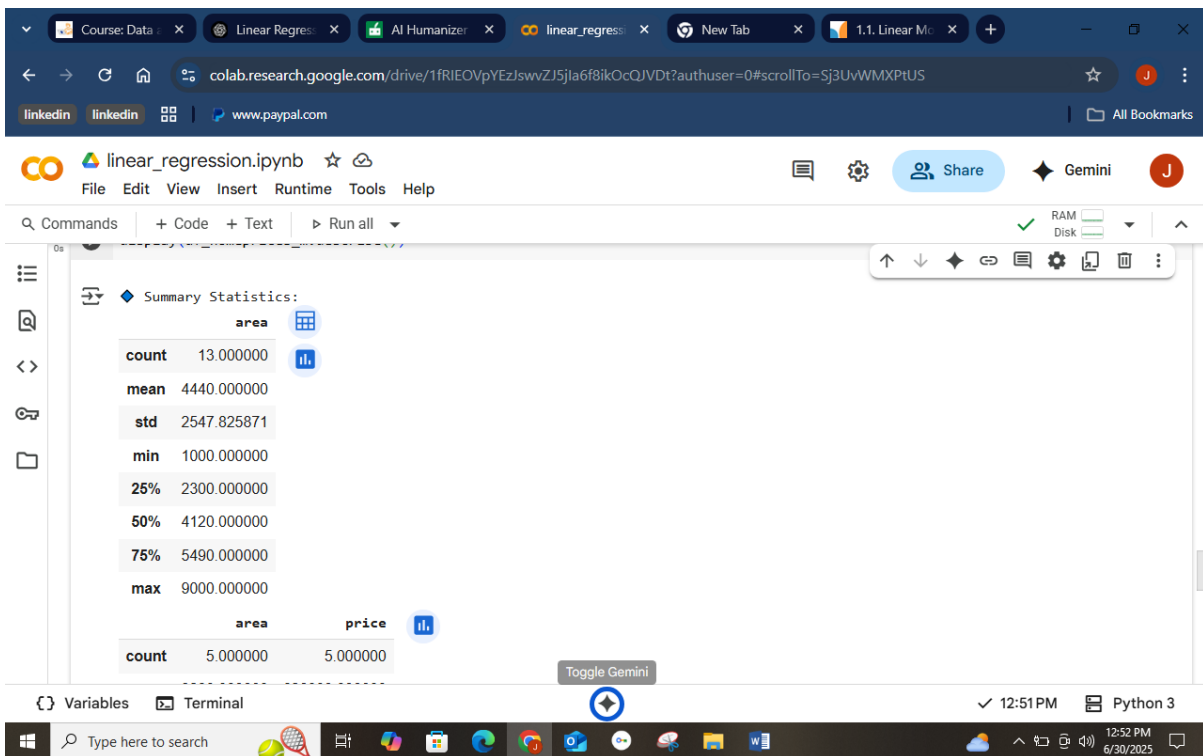
The output of the code is displayed below the code cell:

```
◆ Missing Values:
df_areas:
area    0
dtype: int64

df_homeprices:
area    0
price   0
dtype: int64

df_homeprices-m:
area    0
bedrooms 1
age      0
price    0
dtype: int64
```

Fig 2.1.6: Output of the missing values of the dataset



The screenshot shows the same Google Colab notebook. The code cell contains the following Python code:

```
df_areas.describe()
```

The output of the code is displayed below the code cell:

```
◆ Summary Statistics:
count    13.000000
mean     4440.000000
std      2547.825871
min       1000.000000
25%      2300.000000
50%      4120.000000
75%      5490.000000
max       9000.000000

area    price
count    5.000000  5.000000
```

Fig 2.1.7: Output of summary statistics for area

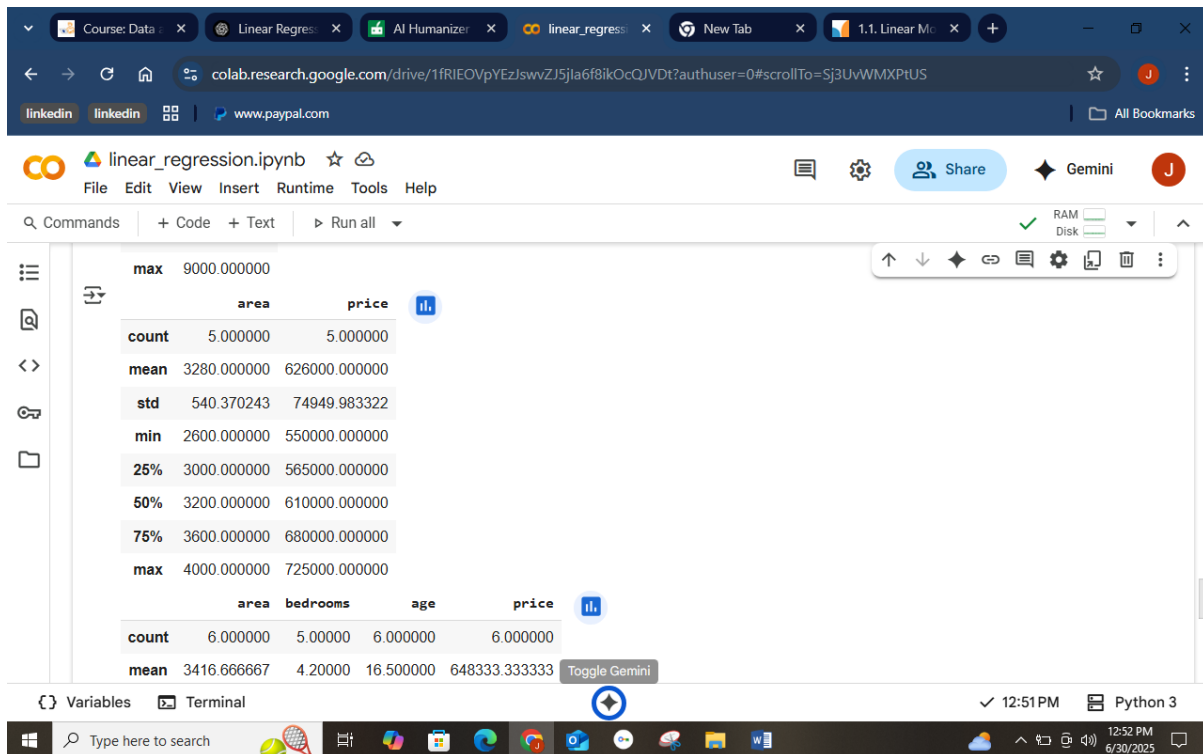


Fig 2.1.8: Output of summary statistics for homeprice

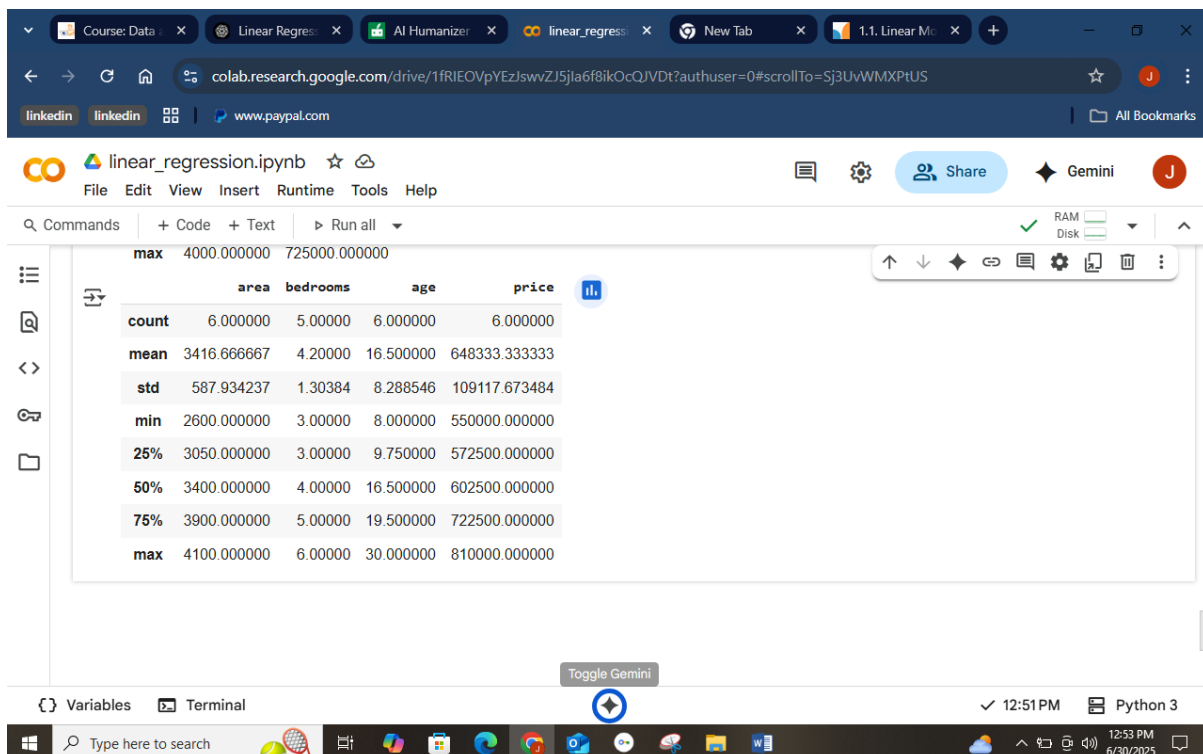


Fig 2.1.9: Output of summary statistics for homeprice_m

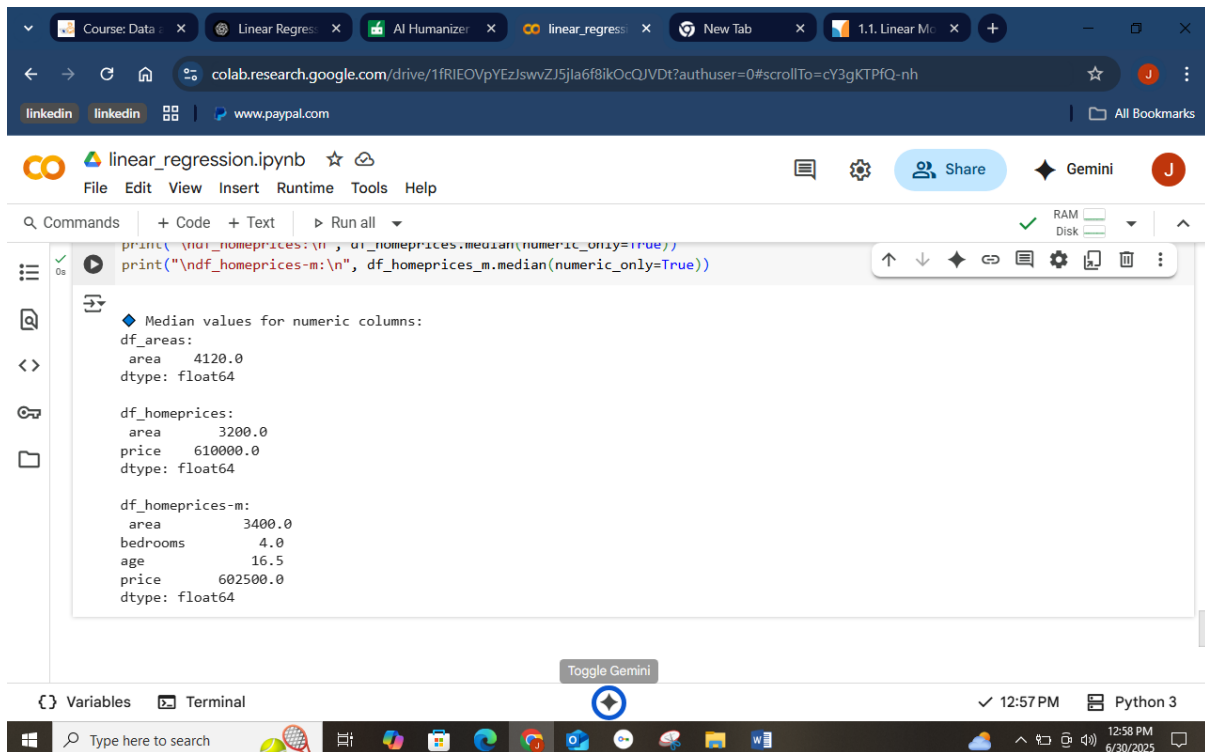


Fig 2.1.10: Output of median for area, homeprice, homeprice_m

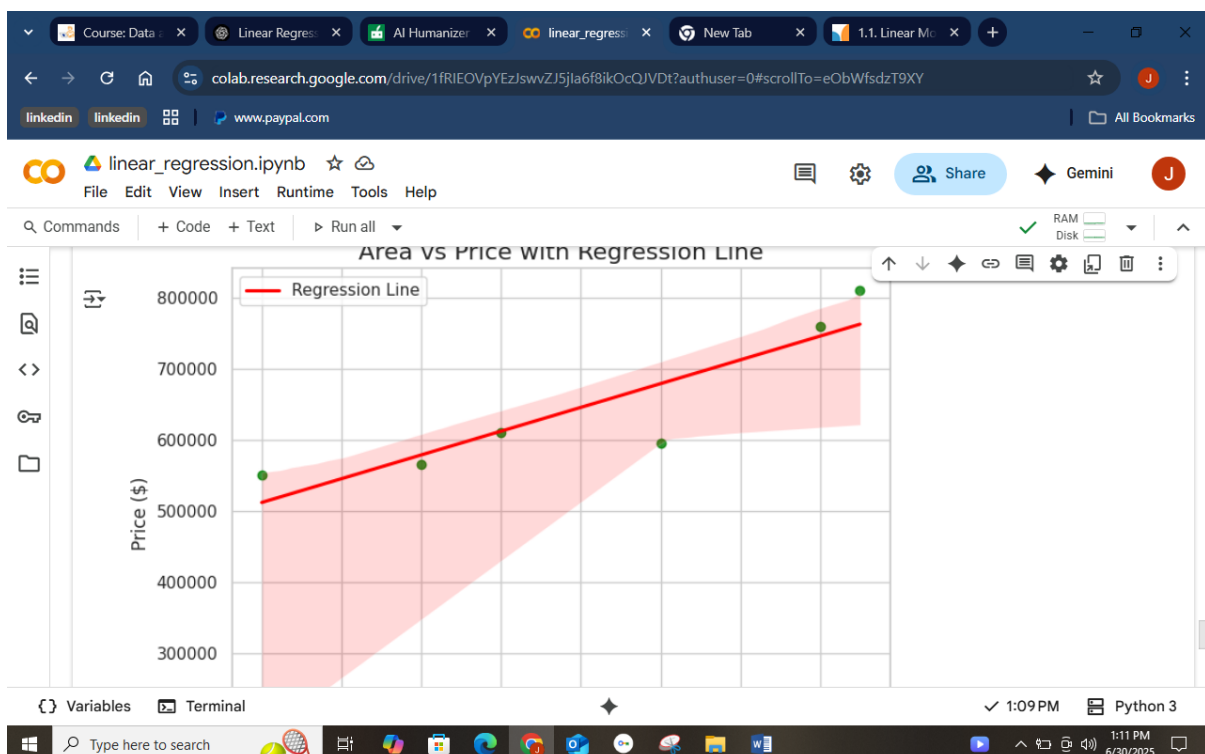


Fig 2.1.10: Output of scatter plot of area vs price with regression line

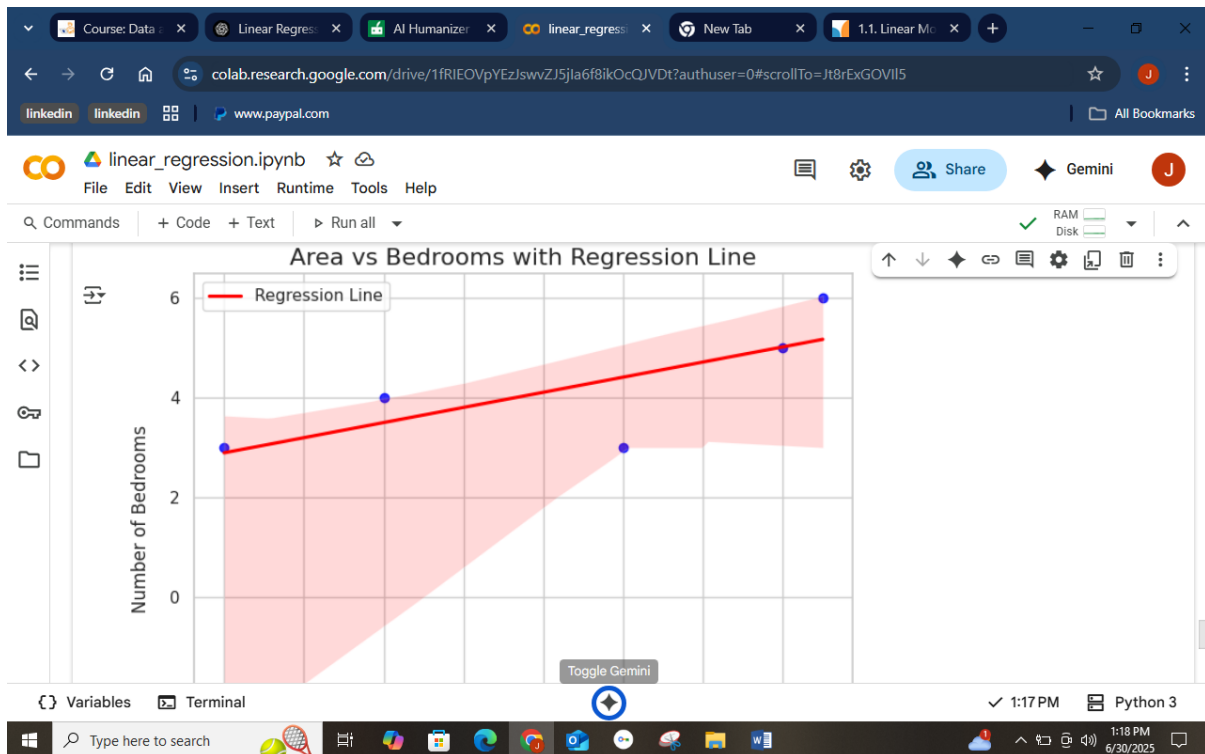


Fig 2.1.11: Output of scatter plot of area vs bedroom with regression line

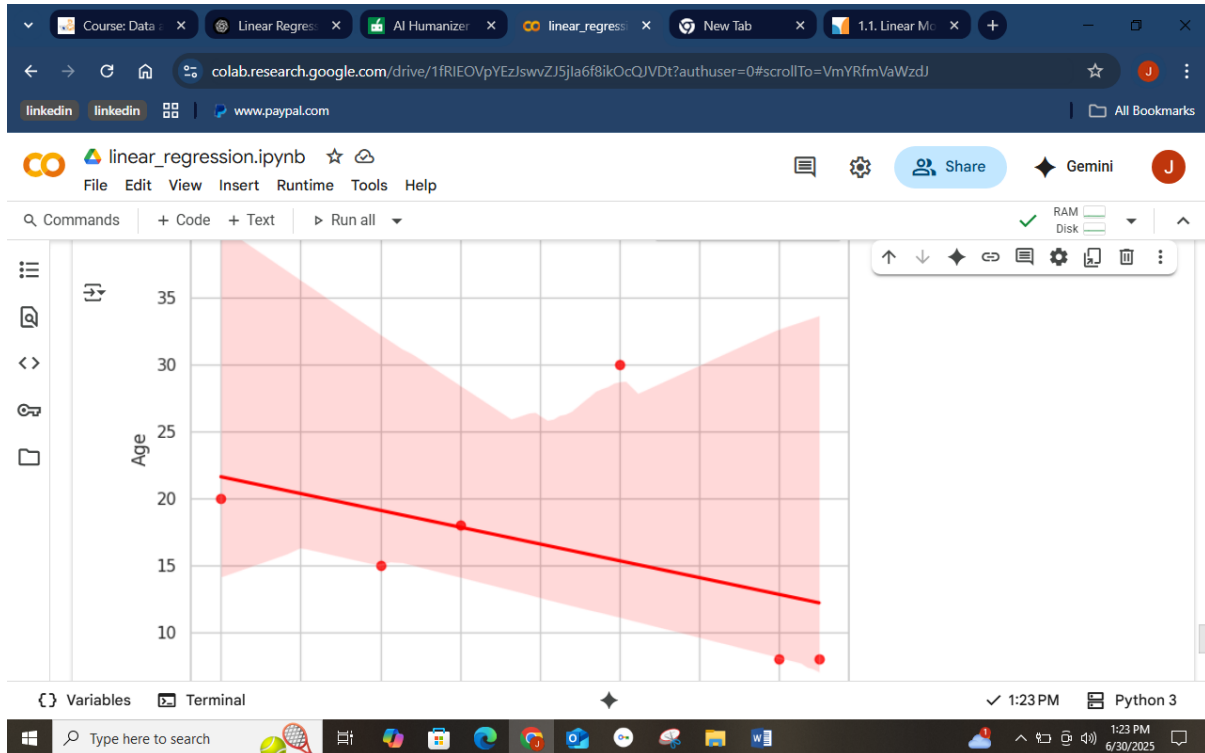
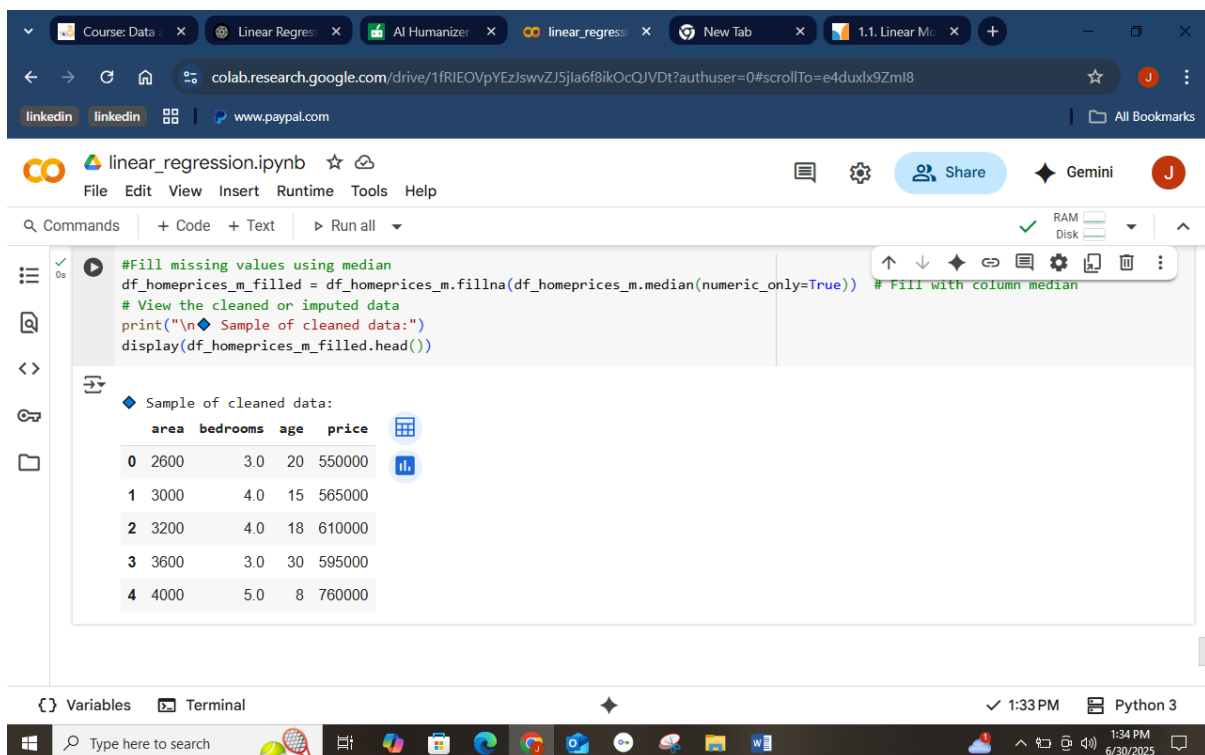


Fig 2.1.12: Output of scatter plot of area vs age with regression line

2.2: Prepare the Data

The dataset is prepared and cleaned before the model is trained. To make sure the model doesn't yield skewed or incorrect results, this entails looking for and managing any missing values. Rows with missing entries are frequently filled in using statistical techniques like mean or median imputation, or they can be removed.

Following cleaning, a standard 70:30 or 80:20 ratio is used to divide the dataset into training and test sets. The linear regression model is fitted using the training set, and its performance on unknown data is assessed using the test set. This stage helps avoid overfitting and is essential for evaluating how well the model generalizes outside of the training set.



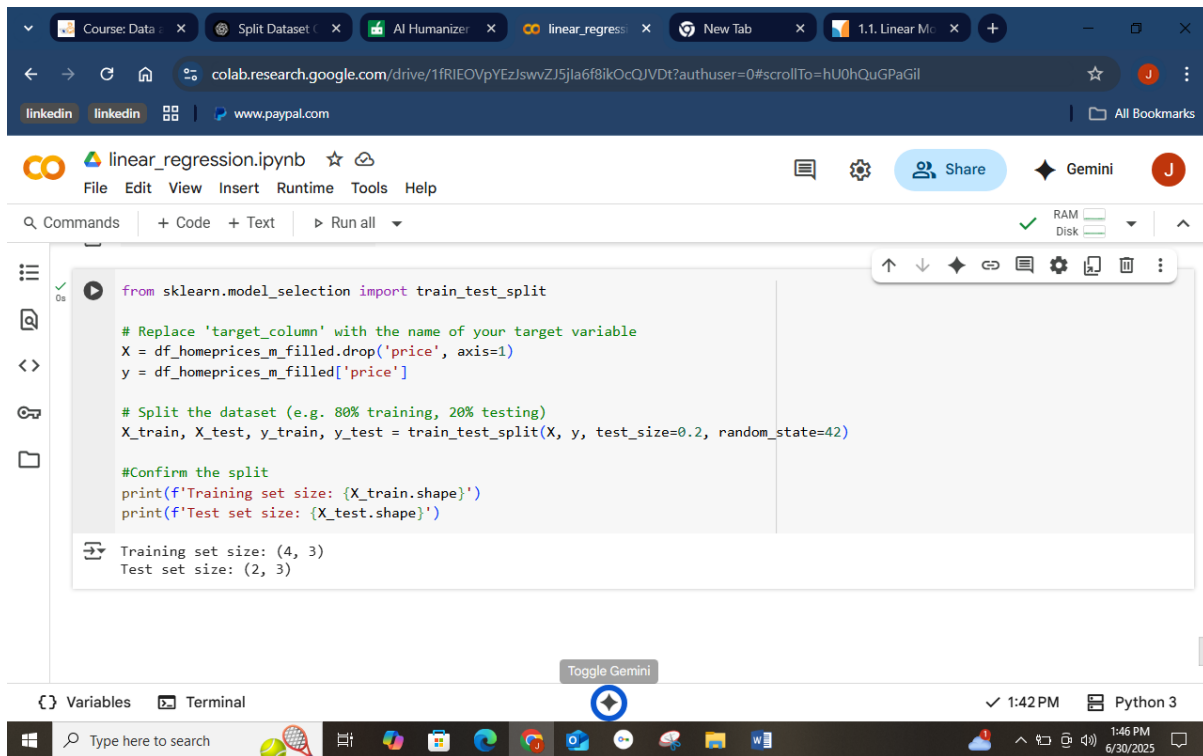
The screenshot shows a Google Colab notebook titled 'linear_regression.ipynb'. The code cell contains the following Python code:

```
#Fill missing values using median
df_homeprices_m_filled = df_homeprices_m.fillna(df_homeprices_m.median(numeric_only=True)) # Fill with column median
# View the cleaned or imputed data
print("\n◆ Sample of cleaned data:")
display(df_homeprices_m_filled.head())
```

The output of the code is a sample of the cleaned data, displayed as a table:

	area	bedrooms	age	price
0	2600	3.0	20	550000
1	3000	4.0	15	565000
2	3200	4.0	18	610000
3	3600	3.0	30	595000
4	4000	5.0	8	760000

Fig 2.2.1: Output of cleaning missing value bedroom in homeprice_m



The screenshot displays a Google Colab notebook titled 'linear_regression.ipynb'. The notebook is open in a web browser with the URL 'colab.research.google.com/drive/1fRIEOVpYEzJswvZJ5Jla6f8IkOcQJVDt?authuser=0#scrollTo=hU0hQuGPaGil'. The notebook interface includes a menu bar (File, Edit, View, Insert, Runtime, Tools, Help) and a toolbar with options like 'Share', 'Gemin', and 'Run all'. The code cell contains the following Python code:

```
from sklearn.model_selection import train_test_split

# Replace 'target_column' with the name of your target variable
X = df_homeprices_m_filled.drop('price', axis=1)
y = df_homeprices_m_filled['price']

# Split the dataset (e.g. 80% training, 20% testing)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Confirm the split
print(f'Training set size: {X_train.shape}')
print(f'Test set size: {X_test.shape}')
```

The output of the code is displayed below the code cell:

```
Training set size: (4, 3)
Test set size: (2, 3)
```

The bottom of the notebook shows the 'Variables' and 'Terminal' tabs, and the system tray at the bottom of the browser window.

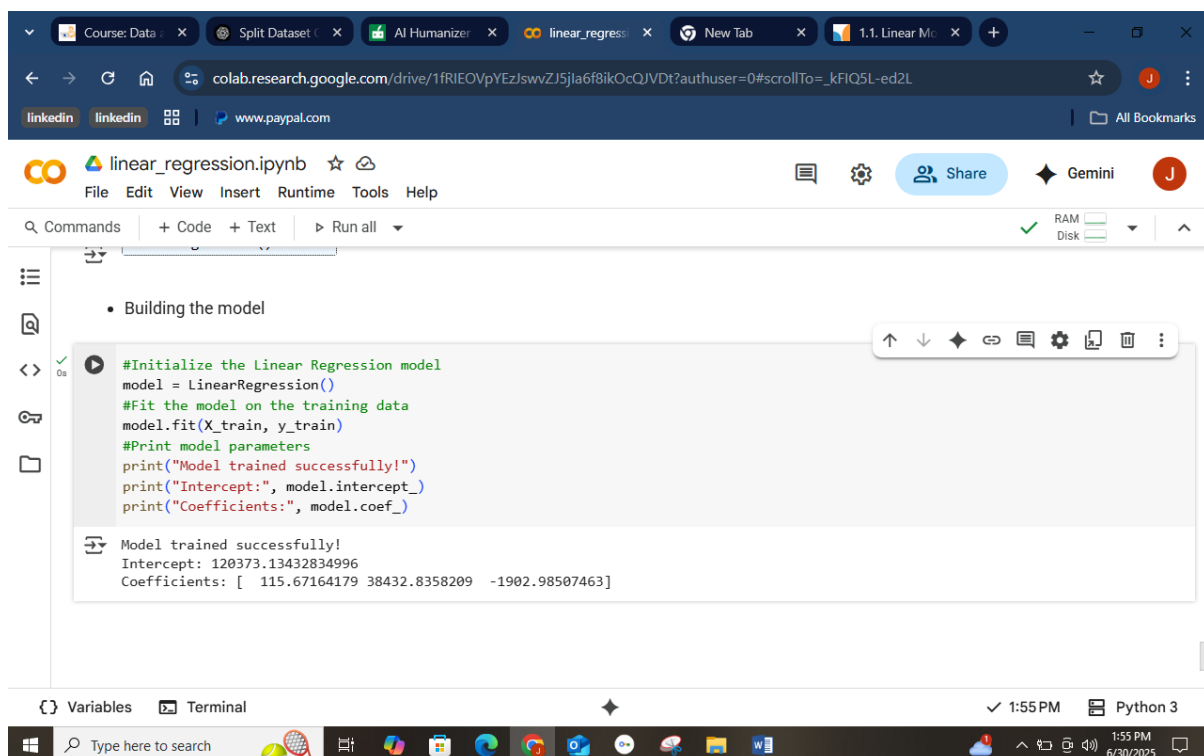
Fig 2.2.2: Output of training the data set in homeprice_m

2.3: Build the Model

With the training data prepared, the next step is to build a **Linear Regression model** using the Linear Regression class from the **Scikit-learn** library. This model aims to find the best-fitting straight line that describes the relationship between the independent variable(s) and the dependent variable.

The model is initialized and then **fitted to the training data** using the `.fit()` method. During this process, the algorithm learns the optimal coefficients (slope and intercept) that minimize the difference between the actual and predicted values based on a least squares approach.

This simple yet powerful model serves as a foundational method in predictive analytics and helps establish a baseline for future, more complex models.



The screenshot shows a Google Colab notebook titled 'linear_regression.ipynb'. The notebook is open to a cell titled 'Building the model'. The code in the cell is as follows:

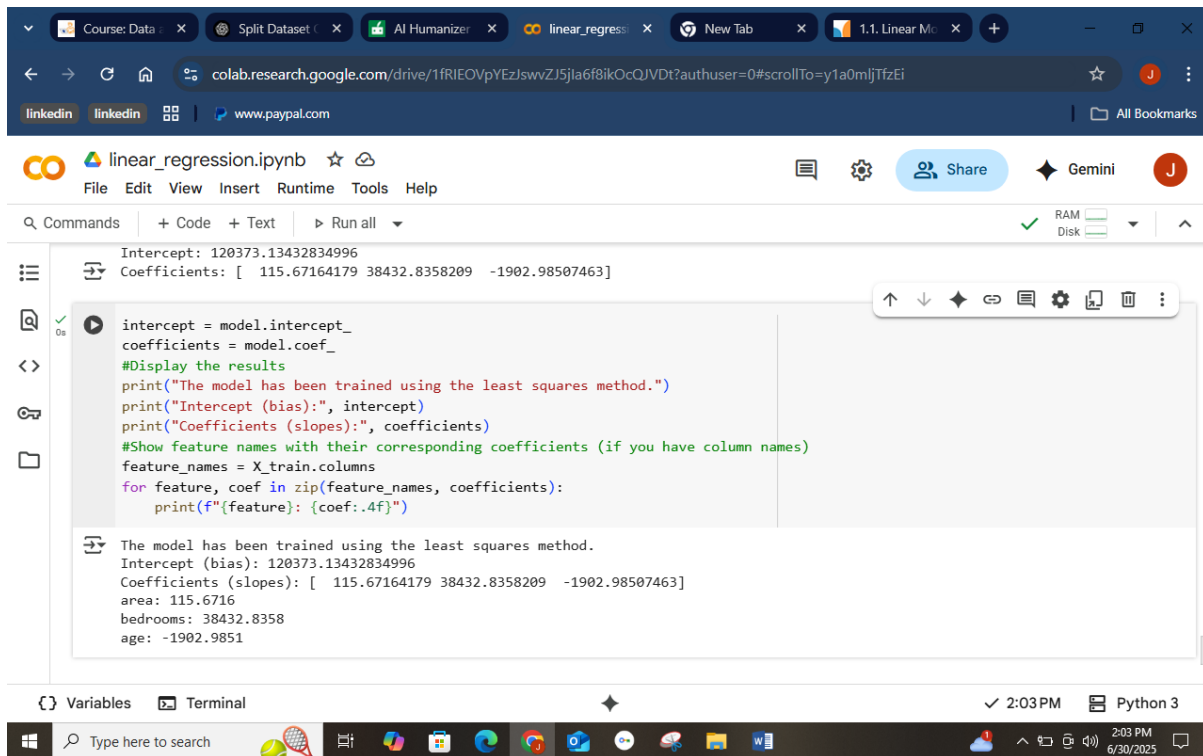
```
#Initialize the Linear Regression model
model = LinearRegression()
#Fit the model on the training data
model.fit(X_train, y_train)
#Print model parameters
print("Model trained successfully!")
print("Intercept:", model.intercept_)
print("Coefficients:", model.coef_)
```

The output of the cell is displayed below the code:

```
Model trained successfully!
Intercept: 120373.13432834996
Coefficients: [ 115.67164179 38432.8358209 -1902.98507463]
```

The notebook interface includes a menu bar (File, Edit, View, Insert, Runtime, Tools, Help), a toolbar with icons for commands, code, text, and running all cells, and a status bar at the bottom showing the time (1:55 PM) and the Python version (Python 3).

Fig 2.3.1: Output of model trained successfully



The screenshot shows a Google Colab notebook titled 'linear_regression.ipynb'. The notebook is open in a web browser with the URL `colab.research.google.com/drive/1fRIEOVpYfEzJswvZJ5jla6f8ikOcQJVDt?authuser=0#scrollTo=y1a0mJfTfzEi`. The notebook interface includes a menu bar (File, Edit, View, Insert, Runtime, Tools, Help), a toolbar with icons for commands, code, text, and running, and a sidebar with icons for file explorer, search, and help. The main area displays a code cell with the following Python code:

```

intercept = model.intercept_
coefficients = model.coef_
#Display the results
print("The model has been trained using the least squares method.")
print("Intercept (bias):", intercept)
print("Coefficients (slopes):", coefficients)
#Show feature names with their corresponding coefficients (if you have column names)
feature_names = X_train.columns
for feature, coef in zip(feature_names, coefficients):
    print(f"{feature}: {coef:.4f}")

```

The output of the code cell is displayed below the code:

```

The model has been trained using the least squares method.
Intercept (bias): 120373.13432834996
Coefficients (slopes): [ 115.67164179 38432.8358209 -1902.98507463]
area: 115.6716
bedrooms: 38432.8358
age: -1902.9851

```

The bottom of the notebook shows a status bar with 'Variables', 'Terminal', and 'Python 3' tabs. The system tray at the bottom of the browser window shows the time as 2:03 PM on 6/30/2025.

Fig 2.3.2: Output of coefficients and intercepts

2.4: Evaluate the Model

After training the linear regression model, its performance is evaluated using standard regression metrics:

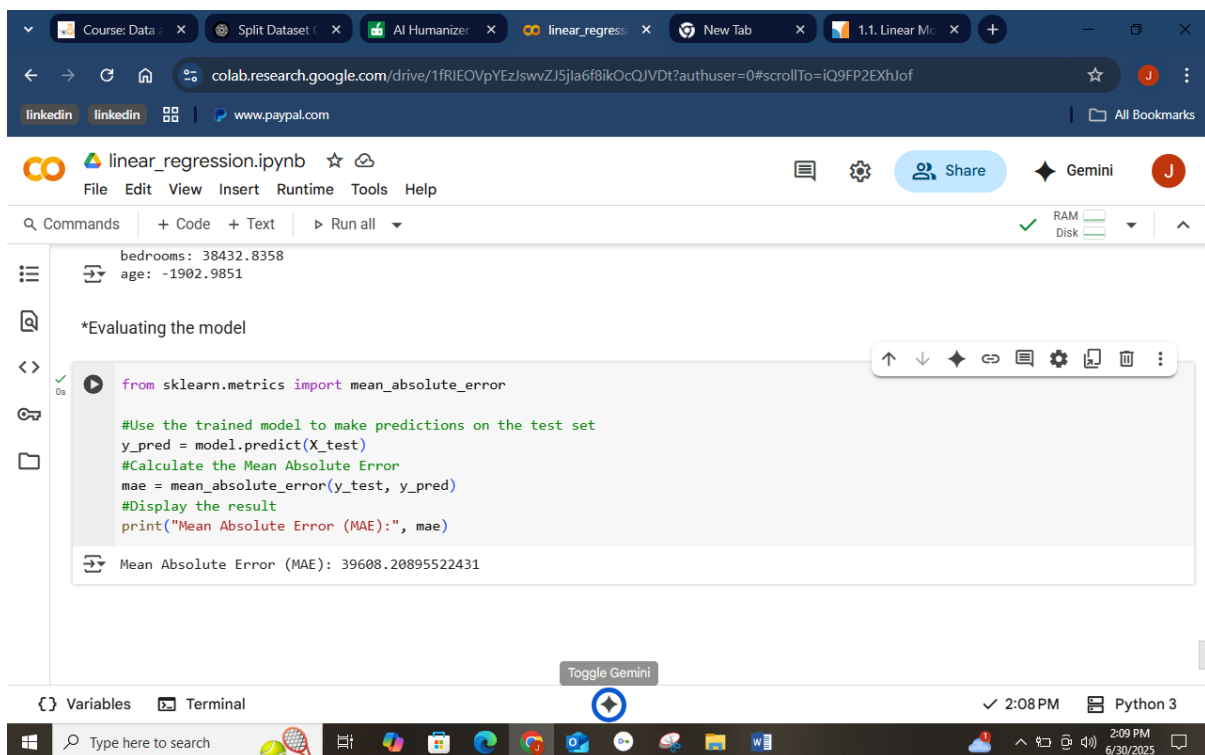
Mean Absolute Error (MAE): Measures the average magnitude of the errors in a set of predictions, without considering their direction.

Mean Squared Error (MSE): Calculates the average squared difference between actual and predicted values. It gives more weight to larger errors.

Root Mean Squared Error (RMSE): The square root of MSE, providing an error metric in the same units as the original data.

R² Score (Coefficient of Determination): Indicates how well the model explains the variance in the target variable. An R² score close to 1 means a strong fit.

These metrics help assess the model's **accuracy** and **generalization ability**, offering insights into how well it is likely to perform on new, unseen data.



The screenshot shows a Google Colab notebook titled 'linear_regression.ipynb'. The notebook is open to a cell titled '*Evaluating the model'. The code in the cell is as follows:

```
from sklearn.metrics import mean_absolute_error

#Use the trained model to make predictions on the test set
y_pred = model.predict(X_test)
#Calculate the Mean Absolute Error
mae = mean_absolute_error(y_test, y_pred)
#Display the result
print("Mean Absolute Error (MAE):", mae)
```

The output of the cell is displayed below the code:

```
Mean Absolute Error (MAE): 39608.20895522431
```

The notebook interface includes a top bar with tabs for 'Course: Data', 'Split Dataset', 'AI Humanizer', 'linear_regres...', 'New Tab', and '1.1. Linear Mo...'. The bottom bar shows the system tray with various icons and the date '6/30/2025'.

Fig 2.4.1: Output of MAE

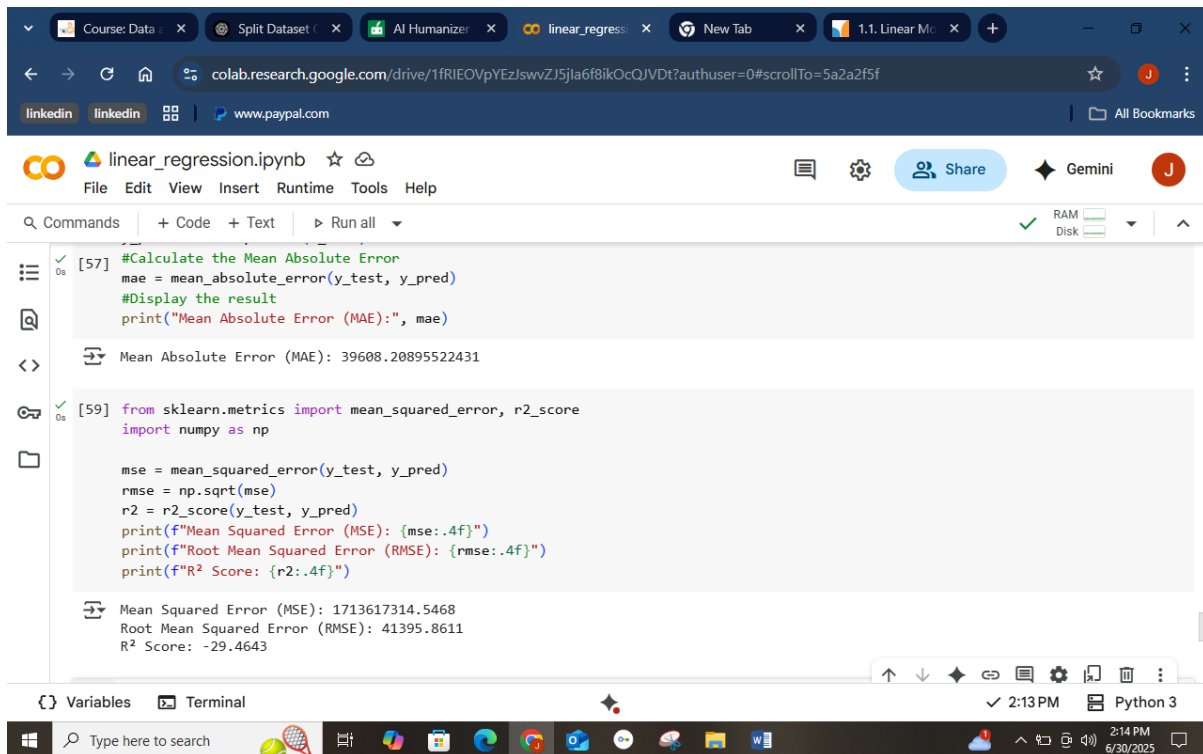


Fig 2.4.2: Output of MSE, RMSE, R2

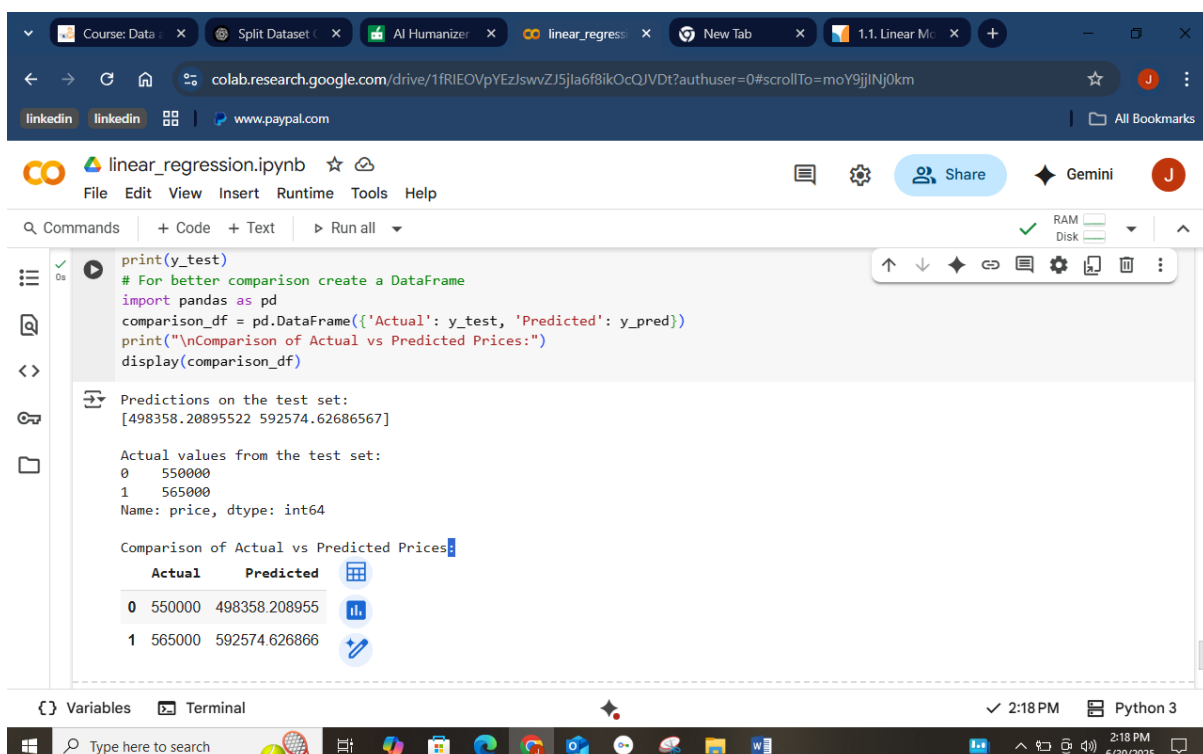


Fig 2.4.3: Output on prediction on the test set

2.5: Visualize the Results

To better understand the model's performance, the regression line is plotted against the actual data points. This visualization shows how well the model captures the underlying relationship between the independent and dependent variables.

A **scatter plot** is used to display the original data, and the **regression line** — representing the model's predictions — is overlaid to illustrate the fit. If the model is accurate, the line should pass through the data in a way that minimizes the distance between the actual points and the predicted values.

This step not only confirms the presence of a linear relationship but also helps detect any patterns, trends, or outliers that may affect model performance.

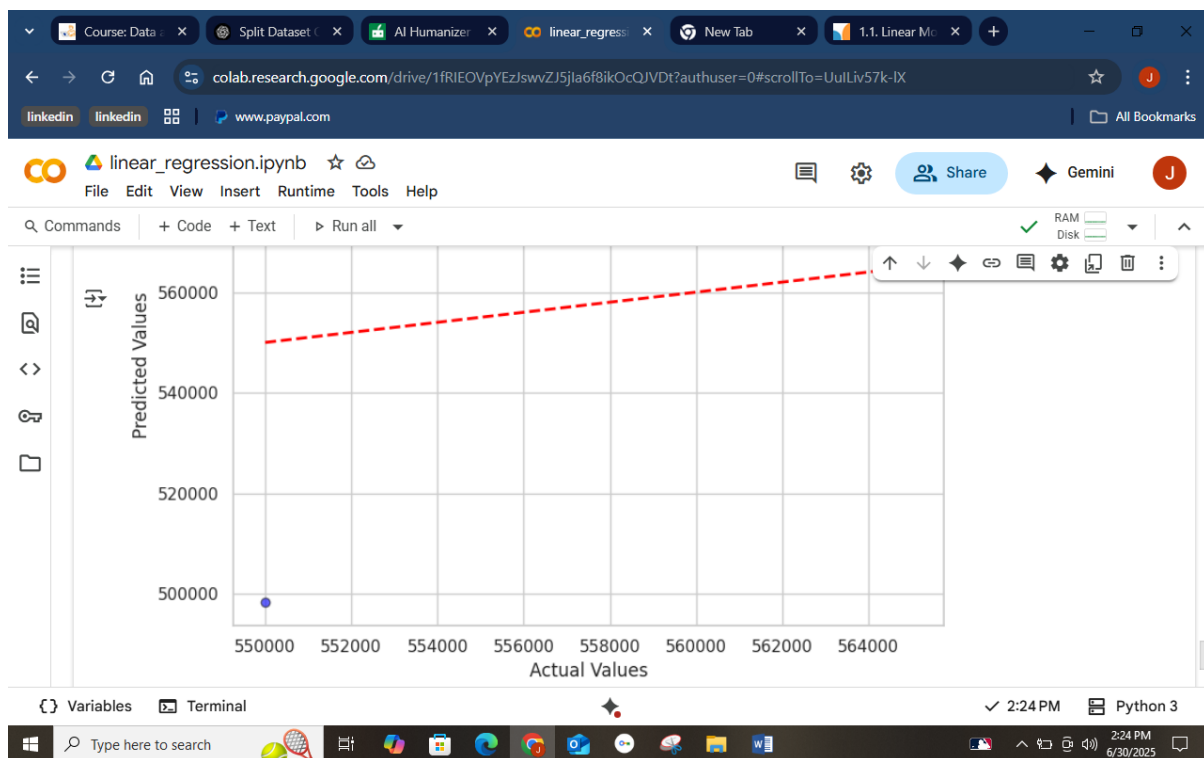


Fig 2.5.1: Output of scatter plot of actual values vs predicted values

- **Blue dots:** Model predictions vs. actual values.
- **Red dashed line:** The "perfect predictions" line — the closer the dots are to this line, the better the model.

Residual Plot

This shows the difference between actual and predicted values (errors). It helps identify patterns your model might be missing

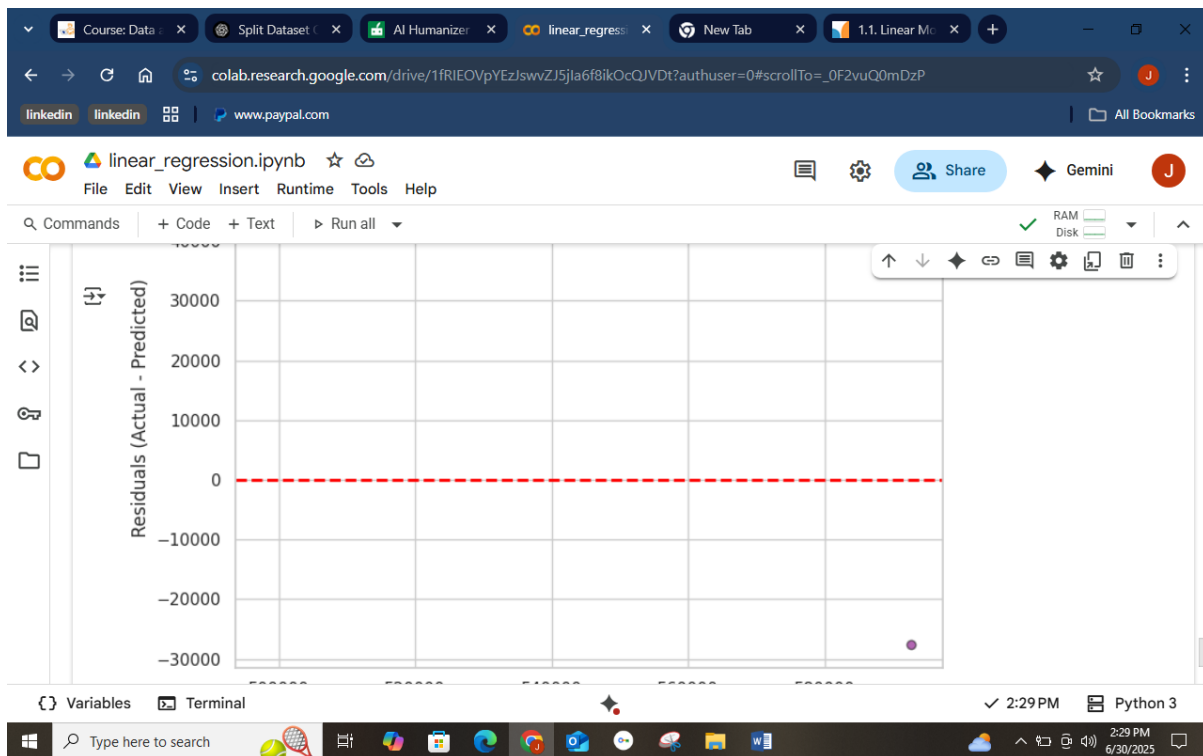


Fig 2.5.2: Output of residuals

In the **residual plot**, points scattering randomly around the red line (zero) suggest problems like non-linearity.

Distribution of Prediction Errors

This checks if the errors are normally distributed — a good sign that your model fits well.

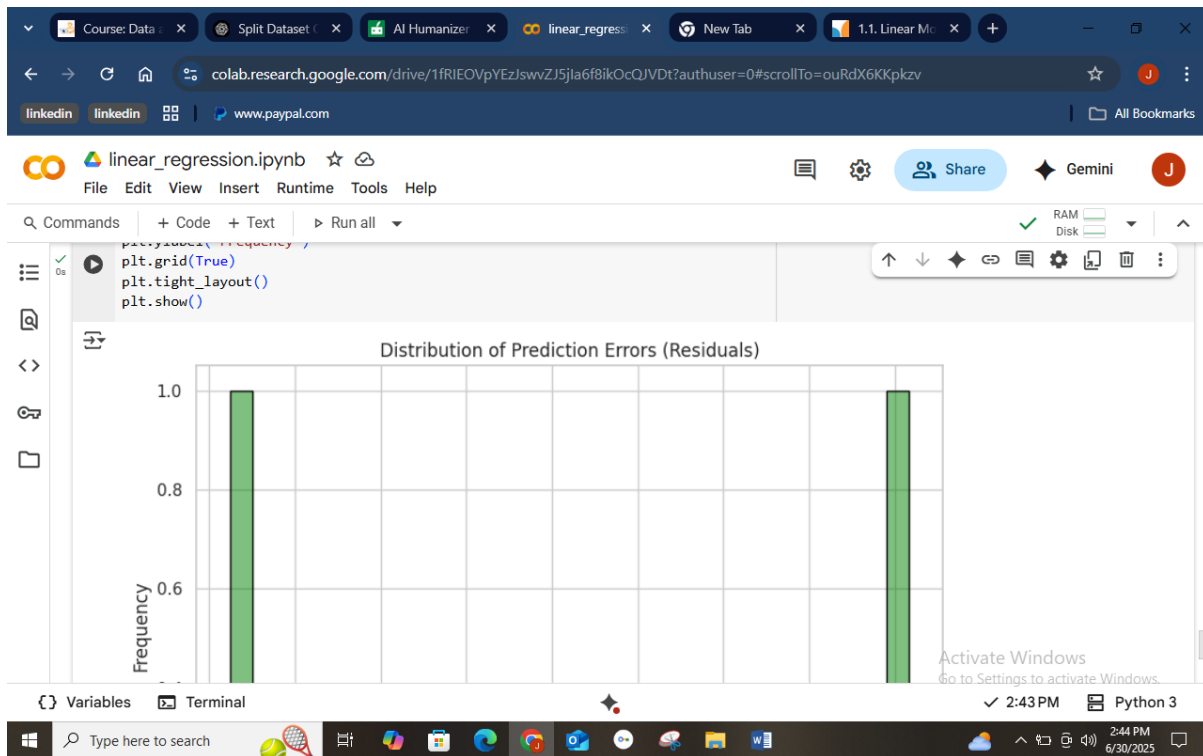


Fig 2.5.3: Output of distribution of prediction errors

2.6: Share Your Work

To showcase this project as part of a growing data science portfolio, I have uploaded it to my **GitHub** and **Google Colab**. Sharing the project publicly demonstrates my ability to work with real-world data and apply machine learning techniques effectively.

The repository or notebook includes all key components: data loading, preprocessing, model building, evaluation, and visualization. A clear markdown documentation is also provided to explain the project's purpose and guide viewers through the workflow.

Link to Code:

<https://colab.research.google.com/drive/1fRiEOVpYEzJswvZJ5jIa6f8ikOcQJVDt?usp=sharing>

Conclusion

This project provided a comprehensive, hands-on experience with implementing a **Linear Regression model** using Python. Starting from loading and exploring a real-world dataset, I gained insights into the structure and relationships within the data through descriptive statistics and visualizations. I then prepared the dataset by handling any missing values and splitting it into training and testing subsets to ensure the model could be evaluated fairly on unseen data.

The evaluation phase involved key performance metrics such as **Mean Absolute Error (MAE)**, **Mean Squared Error (MSE)**, **Root Mean Squared Error (RMSE)**, and the **R² Score**, all of which provided valuable information on the model's accuracy and reliability.

Furthermore, visualizing the regression line against the actual data helped confirm the linearity of the relationship and highlighted how well the model fit the data. These plots also served as a useful tool for identifying outliers and understanding the distribution of errors.

In conclusion, this project not only reinforced fundamental machine learning concepts but also demonstrated the importance of data exploration, cleaning, and model evaluation in the predictive modeling process. It stands as a valuable addition to a data science portfolio and can serve as a foundation for more complex regression and machine learning tasks in future projects.